

Manual Técnico API de Ventas

Carlos Raúl Alberto López Peláez

Quetzaltenango 27 de septiembre del 2025

Índice

1. Introducción al sistema.....	3
1.1. Descripción del Sistema.....	3
1.2. Stack Tecnológico.....	3
2. Configuración de Inicio.....	3
2.1. Requisitos Previos de Software.....	3
2.2. Proceso de Instalación.....	3
2.2.1. Clonar los repositorios.....	3
2.2.2. Configuración del Backend.....	3
2.2.3. Configuración del frontend.....	4
2.2.4. Configuración dentro del backend para la base de datos.....	4
3. Base de Datos.....	5
3.1. Análisis de la base de datos.....	5
3.2. Mejoras para la base de datos.....	5
3.3. Diagrama de la base de datos.....	6
3.4. Consulta de la base de datos.....	6

1. Introducción al sistema

1.1. Descripción del Sistema

Un programa sencillo de filtrado de ventas por categorías para el año 2019 consumo de API usando como backend nestjs y frontend vuejs.

1.2. Stack Tecnológico

Frontend : Vue 3.5.22 , Vuetify 3.10.3.

Backend : NestJS 11.1.6

Node.js: v22.18.0

npm: 11.6.1

2. Configuración de Inicio

2.1. Requisitos Previos de Software

Tener instalado nodejs v22.18.0

Tener instalado npm v11.6.1

Tener instalado mysql 8.4.6

2.2. Proceso de Instalación

2.2.1. Clonar los repositorios

El primer paso es clonar tanto el repositorio del backend como del frontend para hacerlo sólo son necesarios los siguiente comandos:

Frontend

git clone <https://github.com/CralpCode/PruebaTecnica-Frontend.git>

Backend

git clone <https://github.com/CralpCode/PruebaTecnica-Backend.git>

2.2.2. Configuración del Backend

Para instalar el backend es necesario abrir la terminal y ejecutar el siguiente comando (tener en cuenta que se debe de estar dentro de la carpeta de proyecto) :

```
npm install
```

Los comandos para iniciar el proyecto son los siguientes:

```
# development
npm run start

# watch mode
npm run start:dev

# production mode
npm run start:prod
```

cualquiera de estos comandos permite iniciar el backend

2.2.3. Configuración del frontend

Al igual que el backend para el frontend se debe usar el comando:

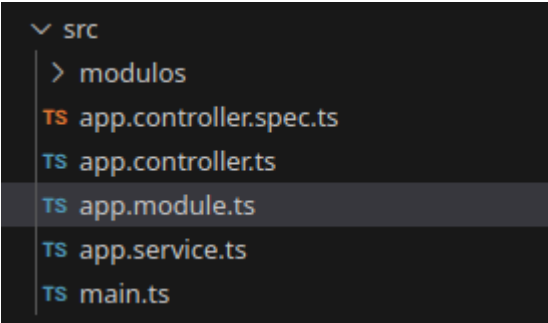
```
npm install
```

Comando para levantar el proyecto:

```
npm run dev
```

2.2.4. Configuración dentro del backend para la base de datos

Dentro del proyecto en la carpeta src hay un archivo llamado app.module.ts



```
▼ src
  > modulos
  TS app.controller.spec.ts
  TS app.controller.ts
  TS app.module.ts
  TS app.service.ts
  TS main.ts
```

Dentro de este archivo está la siguiente configuración :

```
TypeOrmModule.forRoot({  
  type: 'mysql',  
  host: 'localhost',  
  port: 3306,  
  username: 'carlos',  
  password: '12345678',  
  database: 'PruebaTecnica',  
  entities: [Producto, Categoria, Venta],  
  synchronize: true,  
}),
```

Username y password deben ser cambiados por los valores que usted tenga configurados para mysql.

3. Base de Datos

3.1. Análisis de la base de datos

La tabla producto tiene una clave foránea que referencia a categoría, lo que representa una relación de muchos a uno, es decir que muchos productos pueden pertenecer a una misma categoría, pero cada producto pertenece a una sola categoría.

La tabla venta contiene una clave foránea que referencia a producto, lo que establece una relación de muchas ventas para un mismo producto. En este modelo, cada venta está asociada a un solo producto.

En todas las tablas se añade el atributo estado para determinar la vigencia o disponibilidad de los registros, sin necesidad de eliminarlos de la base de datos.

Se añadió ON DELETE RESTRICT a la llave foránea para el código de la categoría y para el código de producto esto para evitar eliminaciones accidentales de datos relacionados.

Además, se utiliza ON UPDATE CASCADE para que cualquier cambio en los identificadores clave se refleje automáticamente en las tablas relacionadas, manteniendo la integridad referencial entre las entidades.

3.2. Mejoras para la base de datos

La base de datos puede mejorarse con atributos adicionales como el precio del producto y la cantidad vendida.

Una de las mejoras sería la creación de una tabla detalle_venta, la cual permitiría representar una relación de muchos a muchos entre ventas y productos. Esta tabla incluiría atributos como la cantidad de cada producto vendido y su precio en el momento de la transacción.

Las consultas pueden mejorar si usamos vistas siempre y cuando sean consultas que no necesiten de parámetros, en el caso de necesitar parámetros se puede llegar a utilizar procedimientos almacenados o funciones en el caso de que se quieran devolver ciertas variables.

La base de datos puede mejorar pero también depende de la lógica del negocio y la complejidad con la que se quiera manejar, además de separar la lógica en el backend y dejar lo demás a la base de datos, de ser necesario incluso la implementación de triggers.

3.3. Diagrama de la base de datos



3.4. Consulta de la base de datos

Desde mysql se puede hacer la siguiente consulta

```
SELECT DISTINCT p.Nombre AS Producto FROM venta v
JOIN producto p ON v.CodigoProducto = p.CodigoProducto
JOIN categoria c ON p.CodigoCategoria = c.CodigoCategoria
WHERE c.Nombre = 'Ropa' AND YEAR(v.Fecha) = 2019;
```

Esto devolverá los productos que fueron comprados en 2019 se usó distinct para que no se repitan los productos comprados, es decir, que aparezca solo una vez laptop por ejemplo que no se repita , ya que queremos saber los productos no todas las ventas.

3.5. Inserciones para la base de datos

```
INSERT INTO categoria (Nombre, Estado) VALUES
('Ropa', true),
('Juguetes', true),
('Zapatos', true),
('Dispositivo Electrónico', true),
('Utiles', true);

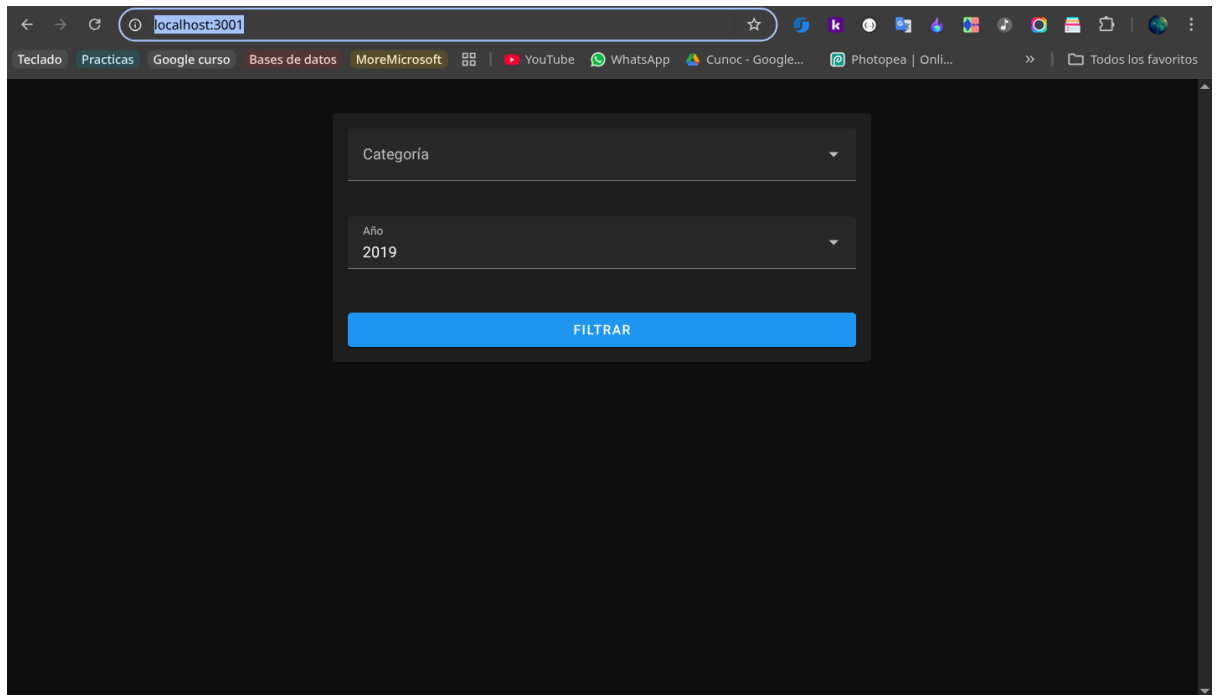
INSERT INTO producto (Nombre, CodigoCategoria, Estado)
VALUES
('Blusa Azul', 1, true),
('Pelota', 2, true),
('Pantalon de Lona', 1, true),
('Telefono', 4, true),
('Lapiz', 5, true),
('Sueter Negro', 1, true),
('Laptop', 4, true),
('Botas cafes', 3, true),
('Television', 4, true),
('Cuaderno', 5, true);

INSERT INTO venta (Fecha, CodigoProducto, Estado) VALUES
('2018-03-05', 1, true),
('2018-03-25', 2, true),
('2018-07-24', 3, true),
('2018-10-05', 4, true),
('2018-11-21', 2, true),
('2018-12-03', 1, true),
('2019-01-30', 5, true),
('2019-03-22', 10, true),
('2019-04-05', 9, true),
('2019-05-25', 8, true),
('2019-07-17', 7, true),
('2019-09-13', 6, true),
('2020-01-19', 1, true),
('2020-02-20', 2, true),
('2020-05-13', 3, true),
('2020-06-19', 9, true),
('2020-07-23', 6, true);
```

4. Endpoints y uso de la aplicación

Para acceder al frontend una vez levantado el servicio acceder a la siguiente url:

<http://localhost:3001/> de devolverle



Estando en esta página y habiendo hecho las configuraciones anteriores ya se puede ejecutar el botón filtrar y debe devolver el listado de productos.

4.1. Endpoints existente

GET <http://localhost:3000/categoria> -> devuelve listado de las categorías

GET <http://localhost:3000/venta/anio> -> devuelve listado de los años de venta

POST <http://localhost:3000/producto/categoria-anio> -> en este hay que enviar un json con {"categoria": "Ropa", "anio": 2019}

4.2. Estructura del backend

El backend esta estuctura de la siguiente en src/modulos/ aqui adentro esta cada uno de los apartados que conectan con la base de datos, es decir las entidades, los servicios , los controladores y dtos.

Las funciones principales se encuentran en los servicios de cada componente, las cuales son:

```
async obtenerCategorias(): Promise<string[]>{
```



```

const resultados = await this.categoriaRepository.find({
  select: ['nombre'],
});

return resultados.map(categoria => categoria.nombre);
}

```

Funcion asincrona obtenerCategorias() simplemente se usa el orm de nestjs para hacer la busqueda de la categoria y se devuelve un array de string que devuelva cada nombre de las categorias y nada mas.

```

async obtenerProductoCategoriaAnio(filterDto :
FilterProductoDto) : Promise<Producto[]> {

  const { categoria, anio } = filterDto;

  const queryBuilder =
this.productoRepository.createQueryBuilder('producto');

  queryBuilder.select('producto.Nombre', 'nombre');

  if (categoria) {
    queryBuilder
      .innerJoin('producto.categoria', 'cat')
      .andWhere('cat.Nombre = :categoria', { categoria });
  }

  if (anio) {
    queryBuilder
      .innerJoin('producto.ventas', 'venta')
      .andWhere('YEAR(venta.Fecha) = :anio', { anio });
  }

  queryBuilder.distinct(true);

  const rawResults = await queryBuilder.getRawMany();

  return rawResults.map(item => item.nombre);
}

```

Funcion asincrona obtenerProductoCategoriaAnio() sigue utilizando el orm de nestjs pero con mayor complejidad se hace un query builder, se selecciona

por nombre de productos y se hace un innerjoin tanto para categoria como para ventas para que se pueda obtener los datos por su categoria y año, ademas de hacer un distinct para que no se repitan los y en esta ocasion devolver los productos solo por su nombre.

```
async obtenerAnioVenta() {
  const query = `
    SELECT DISTINCT EXTRACT(YEAR FROM Fecha) AS year
    FROM venta
    ORDER BY year DESC;
  `;

  const resultados = await
this.ventaRepository.query(query);

  return resultados.map(item => Number(item.year));
}
```

Funcion asincrona obtenerAnioVenta() es una consulta directa para obtener los años de las ventas;

4.3. Estructura del frontend

En el caso del frontend todo se maneja en app.vue esta en scr/app.vue ahi se encuentra la obtencion de los datos usando axios y mostrandolos en un componente list de vue.

5. Documentos

Para los acceder al sql para la base de datos y el pdf de este manual técnico el siguiente link:

<https://github.com/CralpCode/PruebaTecnica-Frontend/tree/main/src/Documentos>

